

FIAP

RELATÓRIO TÉCNICO

Medical Route Optimizer

Otimização de rotas médicas com algoritmo genético e LLM local

Autor: Lucas Amaral

Tech Challenge - Fase 2

Julho de 2026

Documento acadêmico. Dados utilizados no cenário são sintéticos.

Resumo

Este relatório apresenta o desenvolvimento de um sistema de otimização de rotas para distribuição de medicamentos e insumos hospitalares. A solução modela uma variação do Vehicle Routing Problem (VRP) com múltiplos veículos, prioridades de entrega, capacidade de carga e autonomia máxima. A busca por boas soluções é realizada por um algoritmo genético com seleção por torneio, Ordered Crossover, mutação por troca e elitismo. A qualidade das soluções é medida por uma função objetivo que combina distância geodésica, prioridade das entregas e penalidades para violações de carga e autonomia. O algoritmo foi comparado ao baseline do vizinho mais próximo e avaliado em três configurações experimentais. A aplicação possui interface em Streamlit, visualização cartográfica com Folium, testes automatizados e integração com uma LLM local via Ollama para geração de relatórios operacionais e respostas em linguagem natural.

Palavras-chave: algoritmo genético; otimização combinatória; roteamento de veículos; logística hospitalar; LLM; Ollama.

Sumário

1. Introdução
 2. Objetivos
 3. Definição do problema e cenário
 4. Arquitetura da solução
 5. Metodologia do algoritmo genético
 6. Baseline e estratégia experimental
 7. Resultados
 8. Interface, mapa e visualizações
 9. Integração com LLM
 10. Testes e qualidade de software
 11. Limitações e melhorias futuras
 12. Conclusão
- Referências

1. Introdução

A distribuição de medicamentos e insumos hospitalares exige planejamento cuidadoso. Rotas longas aumentam o esforço operacional, enquanto decisões que deixam entregas críticas para o final podem comprometer o nível de serviço. Além disso, veículos possuem limites de capacidade e autonomia que precisam ser respeitados.

O problema cresce rapidamente conforme novas entregas são adicionadas. Para dez destinos, existem $10!$ ordens possíveis antes mesmo de considerar a divisão entre veículos. Em cenários maiores, testar todas as combinações deixa de ser uma estratégia prática. Por esse motivo, o projeto utiliza um algoritmo genético, técnica de computação evolutiva capaz de explorar grandes espaços de busca e produzir soluções de boa qualidade sem enumerar todas as possibilidades.

O sistema desenvolvido combina ciência da computação e engenharia de software: o algoritmo realiza a otimização; a aplicação organiza dados, valida restrições, executa experimentos, apresenta visualizações e integra uma LLM como camada de interpretação textual.

2. Objetivos

2.1 Objetivo geral

Desenvolver um sistema em Python capaz de otimizar rotas de distribuição médica utilizando algoritmo genético, considerando distância, prioridade, capacidade, autonomia e múltiplos veículos, com visualização em mapa e integração com uma LLM.

2.2 Objetivos específicos

- Modelar entregas, veículos e depósito com estruturas de domínio tipadas.
- Representar rotas como cromossomos válidos, sem duplicação ou perda de entregas.
- Implementar seleção, crossover, mutação, elitismo e ciclo evolutivo completo.
- Construir uma função objetivo capaz de equilibrar distância e restrições operacionais.
- Comparar a abordagem evolutiva com um baseline simples.
- Executar experimentos reproduzíveis com diferentes parâmetros.
- Apresentar métricas, gráfico de convergência, rotas por veículo e mapa interativo.
- Utilizar uma LLM local para produzir instruções operacionais e responder perguntas.
- Criar testes automatizados e documentação de execução.

3. Definição do problema e cenário

O projeto trata uma variação do Vehicle Routing Problem. A solução recebe um depósito central, dez entregas e duas vans hospitalares. Cada entrega possui identificador, nome, coordenadas, demanda em quilogramas, prioridade e descrição do item. Cada veículo possui capacidade máxima e autonomia estimada.

Destino	Demanda (kg)	Prioridade	Item
Unidade Centro	12	Crítica	Insulina e medicamentos refrigerados

Unidade Trindade	18	Alta	Antibióticos
Unidade Agrônômica	15	Média	Materiais para curativo
Unidade Estreito	20	Alta	Medicamentos de emergência
Unidade Itacorubi	10	Média	Soro fisiológico
Unidade Lagoa	14	Baixa	Materiais administrativos
Unidade Saco Grande	16	Crítica	Hemoderivados
Unidade Canasvieiras	22	Média	Kits de coleta
Unidade Campeche	17	Alta	Vacinas
Atendimento Domiciliar Sul	8	Baixa	Insumos para home care

Tabela 1 - Entregas sintéticas utilizadas no cenário.

Veículo	Capacidade	Autonomia máxima
Van Hospitalar 01	80 kg	130 km
Van Hospitalar 02	80 kg	130 km

Tabela 2 - Frota disponível.

A demanda total é de 152 kg, enquanto a frota possui capacidade conjunta de 160 kg. Portanto, o cenário é factível em carga total, mas a ordem do cromossomo influencia como as entregas serão distribuídas entre as vans.

4. Arquitetura da solução

A aplicação foi dividida em módulos independentes. Os dados são carregados de arquivos JSON e validados antes da otimização. O algoritmo genético e o baseline compartilham a mesma função objetivo, permitindo comparação coerente. O resultado alimenta a interface Streamlit, o mapa Folium e o serviço de LLM.

Arquitetura lógica da solução

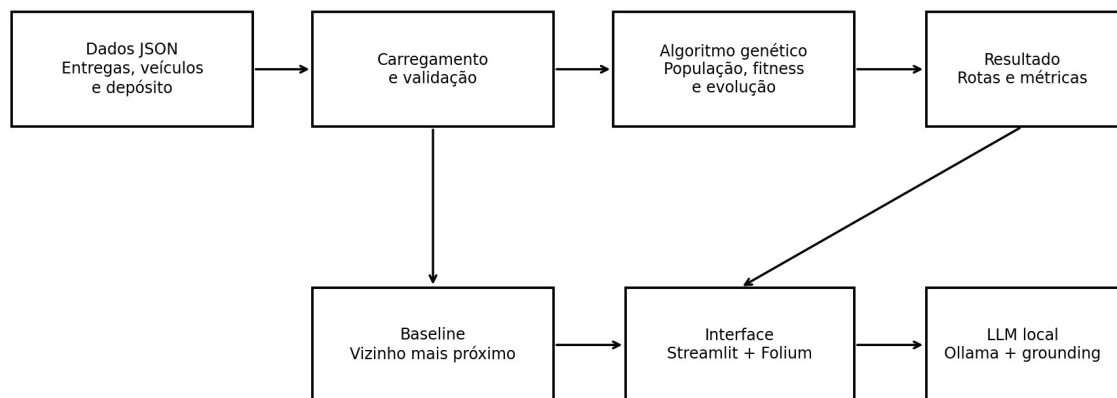


Figura 1 - Arquitetura lógica da solução.

4.1 Organização dos módulos

models.py: Modelos de domínio para depósito, entregas, veículos e rotas.

distance.py: Cálculo de distância por Haversine e distância total da rota.

io_utils.py: Leitura e validação dos arquivos do cenário.

genetic/: População, decodificação, fitness, seleção, crossover, mutação e otimizador.

baseline.py: Implementação do vizinho mais próximo.

map_service.py: Construção do mapa interativo.

llm_service.py: Conversão do resultado em contexto e comunicação com o Ollama.

app.py: Interface principal em Streamlit.

experiments/: Execução reproduzível das configurações experimentais.

tests/: Testes automatizados com pytest.

5. Metodologia do algoritmo genético

5.1 Representação genética

Cada cromossomo é uma permutação dos identificadores das entregas. Todos os genes aparecem exatamente uma vez, garantindo que nenhuma entrega seja duplicada ou omitida.

[1, 4, 9, 10, 3, 7, 2, 5, 8, 6]

O cromossomo representa uma ordem global. Um decodificador sequencial percorre essa ordem e atribui entregas ao veículo atual até que a próxima ultrapasse sua capacidade; então passa para o próximo veículo. Caso a última van seja excedida, a violação é mantida e penalizada, em vez de descartar a entrega.

5.2 Cálculo de distância

A distância entre duas coordenadas é calculada pela fórmula de Haversine, com raio médio da Terra de 6.371,0088 km. Cada rota parte do depósito, visita as entregas na ordem definida e retorna ao depósito. A estimativa é geodésica: não considera ruas, pontes, tráfego ou sentido de circulação.

5.3 Função objetivo

A função objetivo combina distância e penalidades. Quanto menor a pontuação, melhor a solução.

pontuação = distância total
+ 2.000 × excesso de carga (kg)
+ 1.000 × excesso de autonomia (km)
+ penalidade de prioridade

A penalidade de prioridade é calculada pela posição da entrega na rota multiplicada por um peso: 30 para prioridade crítica, 15 para alta, 5 para média e 0 para baixa. Assim, uma

entrega crítica na primeira posição recebe penalidade zero, enquanto a mesma entrega em posição posterior aumenta a pontuação.

A pontuação não representa dinheiro. Trata-se de uma escala abstrata criada para comparar soluções.

5.4 Seleção, crossover, mutação e elitismo

Seleção por torneio: sorteia um subconjunto da população e escolhe o indivíduo de menor custo. O tamanho do torneio controla a pressão seletiva.

Ordered Crossover (OX): preserva um segmento de um pai e completa as posições com a ordem relativa do outro, mantendo uma permutação válida.

Swap Mutation: troca duas posições do cromossomo conforme uma taxa configurável, introduzindo diversidade.

Elitismo: copia diretamente os melhores indivíduos para a geração seguinte, evitando a perda da melhor solução encontrada.

5.5 Ciclo evolutivo

1. Criar população aleatória válida.
2. Avaliar todos os indivíduos.
3. Ordenar pela pontuação.
4. Preservar a elite.
5. Selecionar pais por torneio.
6. Aplicar crossover conforme a taxa configurada.
7. Aplicar mutação nos filhos.
8. Completar a nova população e repetir.

A semente aleatória torna a execução reproduzível: mesmos dados, parâmetros e semente produzem o mesmo resultado.

6. Baseline e estratégia experimental

6.1 Baseline do vizinho mais próximo

O baseline começa no depósito e escolhe repetidamente a entrega não visitada mais próxima da posição atual. É uma heurística gulosa rápida e determinística, mas não reconsidera decisões anteriores e não otimiza diretamente a prioridade. O cromossomo produzido é avaliado pela mesma função objetivo aplicada ao algoritmo genético.

6.2 Configurações experimentais

Experimento	População	Gerações	Mutação	Crossover	Torneio	Elite
Experimento 1	50	100	5%	80%	3	2
Experimento 2	100	200	10%	85%	3	2
Experimento 3	150	300	15%	90%	4	4

Tabela 3 - Configurações do algoritmo genético.

Cada configuração foi executada três vezes com sementes 42, 43 e 44. Foram observados melhor resultado, média das melhores pontuações e distância. O tempo absoluto pode variar conforme o equipamento, portanto a comparação principal concentra-se na qualidade das soluções e no aumento relativo do esforço computacional.

7. Resultados

7.1 Resultado da configuração padrão

Indicador	Resultado
Pontuação da função objetivo	197.723
Distância total estimada	92.723 km
Penalidade de prioridade	105.0 pontos
Excesso de carga	0 kg
Excesso de autonomia	0 km
Geração da melhor solução	33
Redução da pontuação em relação ao baseline	25.01%

Tabela 4 - Indicadores da execução padrão (população 100, 200 gerações, semente 42).

A solução genética alcançou pontuação 197,723. O baseline obteve 263,654. A distância do baseline foi menor (88,654 km), porém sua penalidade de prioridade foi de 175 pontos, contra 105 pontos na solução genética. Isso demonstra o trade-off central do projeto: a melhor solução pela função objetivo não é necessariamente a rota de menor distância, pois também antecipa entregas prioritárias.

7.2 Rotas obtidas

Veículo	Ordem das entregas	Carga	Distância	Violações
Van Hospitalar 01	Unidade Centro → Unidade Estreito → Unidade Campeche → Atendimento Domiciliar Sul → Unidade Agronômica	72.0 kg	33.671 km	Nenhuma
Van Hospitalar 02	Unidade Saco Grande → Unidade Trindade → Unidade Itacorubi → Unidade Canasvieiras → Unidade Lagoa	80.0 kg	59.051 km	Nenhuma

Tabela 5 - Distribuição da melhor solução entre os veículos.

A Van Hospitalar 01 transportou 72 kg e a Van Hospitalar 02 utilizou exatamente sua capacidade de 80 kg. As duas rotas permaneceram abaixo da autonomia máxima de 130 km.

7.3 Comparação dos experimentos

Experimento	Pontuação média	Melhor pontuação	Distância média	Melhor distância
Experimento 1	204.687	197.723	94.687 km	92.723 km
Experimento 2	204.417	197.723	99.417 km	92.723 km
Experimento 3	200.178	197.723	91.845 km	90.089 km

Tabela 6 - Resultados agregados de três execuções por configuração.

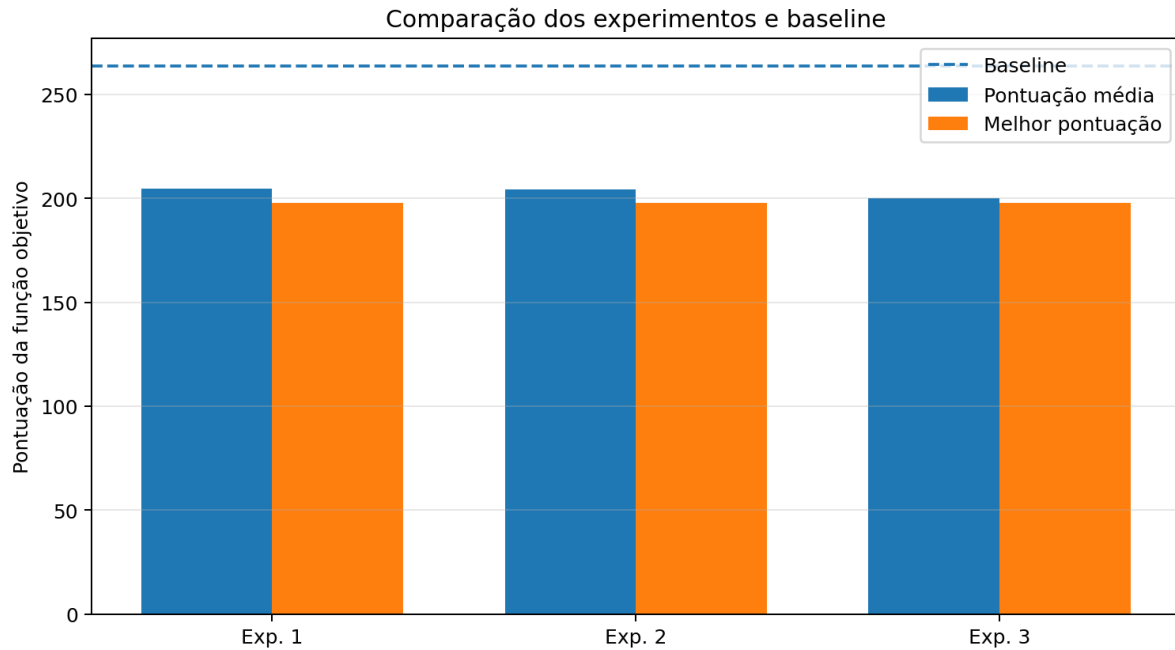


Figura 2 - Comparação das pontuações dos experimentos com o baseline.

Os três experimentos encontraram a melhor pontuação de 197,723 em pelo menos uma execução. O Experimento 3 apresentou a menor pontuação média, indicando maior consistência, mas exige população e número de gerações maiores. O Experimento 1 também encontrou a melhor solução, mostrando que configurações menores podem ser suficientes neste cenário reduzido, embora com maior variabilidade.

7.4 Convergência

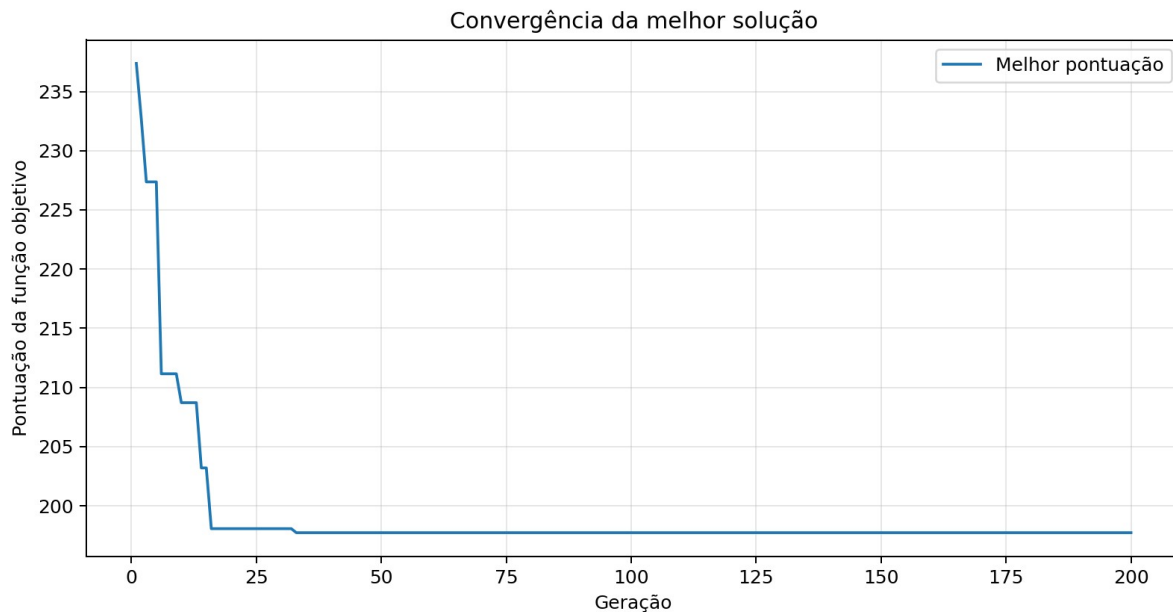


Figura 3 - Evolução da melhor pontuação na configuração padrão.

A melhor pontuação caiu rapidamente nas primeiras gerações e atingiu 197,723 na geração 33. Depois disso, permaneceu estável até o final, comportamento compatível com convergência. O elitismo garante que o melhor valor não aumente nas gerações seguintes.

8. Interface, mapa e visualizações

A interface foi construída com Streamlit para permitir demonstração direta do algoritmo Python. A barra lateral expõe população, gerações, taxas de crossover e mutação, tamanho do torneio, elite e semente. A tela apresenta métricas principais, comparação com o baseline, gráfico de evolução, cromossomo, rotas por veículo e alertas de restrição.

O mapa utiliza Folium e OpenStreetMap. Cada veículo recebe uma linha visual distinta e as entregas são apresentadas com marcadores contendo ordem, item, prioridade e carga. As linhas representam a sequência geográfica das coordenadas, não o caminho real pelas ruas.

9. Integração com LLM

9.1 Papel da LLM

A LLM é utilizada como camada de interpretação, não como mecanismo de otimização. O algoritmo genético calcula e valida as rotas; o resultado é convertido para JSON e enviado a um modelo local executado pelo Ollama. A LLM produz instruções para os motoristas, resumo operacional, alertas e respostas a perguntas em linguagem natural.

Algoritmo genético → resultado estruturado → JSON → prompt com grounding → Ollama → texto

9.2 Modelo e comunicação

O modelo utilizado foi gwen2.5-coder:7b, executado localmente. A aplicação chama o endpoint /api/chat do Ollama com streaming desativado e temperatura baixa, favorecendo

respostas consistentes. A configuração é carregada por variáveis de ambiente e o arquivo .env não é versionado.

9.3 Grounding e controles

- Usar exclusivamente os dados fornecidos no contexto.
- Preservar a ordem exata das paradas.
- Não inventar endereços, horários, motoristas ou medicamentos.
- Não tratar a pontuação da função objetivo como valor monetário.
- Indicar explicitamente quando uma informação não está disponível.
- Destacar entregas críticas e violações de carga ou autonomia.

9.4 Avaliação da resposta

Indicador	Resultado
Cobertura das entregas	100,0%
Rotas com ordem preservada	2/2
Entregas críticas mencionadas	Todas
Violações inventadas	Nenhuma observada no relatório final

Tabela 7 - Avaliação automática do relatório gerado pela LLM.

A avaliação automática procura os nomes das entregas no texto e verifica se sua posição relativa respeita a ordem das rotas. Essa checagem não elimina todos os riscos de alucinação, mas fornece evidência objetiva de cobertura. A revisão humana continua necessária em um contexto real.

10. Testes e qualidade de software

O projeto utiliza pytest e mantém testes separados da aplicação. A suíte cobre cálculo de distância, carregamento e validação dos dados, população, decodificação, fitness, seleção, crossover, mutação, ciclo evolutivo, baseline, mapa e serviço de LLM.

A chamada HTTP ao Ollama é substituída por mocks nos testes automatizados. Isso evita dependência do modelo real durante a suíte e mantém os testes rápidos e reproduzíveis. A integração real é validada manualmente pela aplicação.

Outras práticas adotadas incluem modelos imutáveis com dataclasses congeladas, tipagem, validação de parâmetros, sementes aleatórias, separação de responsabilidades e tratamento controlado de erros de conexão, timeout e resposta inválida da LLM.

11. Limitações e melhorias futuras

11.1 Limitações

- As distâncias são geodésicas e não consideram a malha viária.
- Os dados do cenário são sintéticos.
- O decodificador distribui entregas sequencialmente e não explora todas as divisões possíveis entre veículos.
- O algoritmo genético é heurístico e não garante o ótimo global.

- Não existem janelas de horário, tempo de atendimento ou trânsito em tempo real.
- Não há modelagem específica de cadeia refrigerada além da prioridade e descrição dos itens.
- A LLM pode produzir inconsistências mesmo com grounding e avaliação automática.

11.2 Melhorias futuras

- Integrar OSRM, GraphHopper ou outra engine para rotas reais por ruas.
- Adicionar janelas de entrega e tempos de serviço.
- Permitir múltiplos depósitos e tipos heterogêneos de veículos.
- Criar um cromossomo com separadores ou um decodificador capaz de otimizar a divisão entre veículos.
- Comparar com simulated annealing, busca tabu, OR-Tools ou programação inteira.
- Persistir histórico de execuções e exportar relatórios em PDF.
- Adicionar reotimização dinâmica em caso de atraso ou indisponibilidade.
- Fortalecer a avaliação da LLM com saída estruturada e validação por schema.

12. Conclusão

O projeto implementou com sucesso um sistema completo de otimização de rotas médicas. A solução combina um algoritmo genético funcional, restrições de carga e autonomia, priorização de entregas, comparação com baseline, experimentos, visualizações e integração com uma LLM local.

Na configuração padrão, o algoritmo reduziu a pontuação da função objetivo de 263,654 no baseline para 197,723, sem excessos de carga ou autonomia. A melhoria ocorreu mesmo com uma distância ligeiramente maior, pois a solução evolutiva reduziu significativamente a penalidade causada pelo atraso de entregas prioritárias.

A arquitetura separa claramente cálculo e linguagem: o algoritmo decide as rotas e a LLM explica o resultado. Essa divisão torna a solução mais auditável e reduz o risco de delegar decisões operacionais a um modelo generativo. O sistema atende aos objetivos acadêmicos do desafio e estabelece uma base extensível para cenários logísticos mais realistas.

Referências

FIAP. Tech Challenge - Fase 2: Inteligência Artificial e Algoritmos Genéticos. Documento de especificação do projeto, 2026.

HOLLAND, John H. Adaptation in Natural and Artificial Systems. University of Michigan Press, 1975.

GOLDBERG, David E. Genetic Algorithms in Search, Optimization and Machine Learning. Addison-Wesley, 1989.

TOTH, Paolo; VIGO, Daniele. Vehicle Routing: Problems, Methods, and Applications. SIAM, 2014.

OLLAMA. Plataforma de execução local de modelos de linguagem. Utilizada para integração da LLM no projeto.

OPENSTREETMAP CONTRIBUTORS. Dados cartográficos utilizados como camada visual no mapa da aplicação.

Apêndice A - Execução do projeto

```
python -m venv .venv
source .venv/Scripts/activate
python -m pip install -r requirements.txt
cp .env.example .env
python -m pytest -q
python experiments/run_experiments.py
python -m streamlit run app.py
```

Configuração recomendada da LLM:

```
OLLAMA_BASE_URL=http://127.0.0.1:11434
OLLAMA_MODEL=qwen2.5-coder:7b
OLLAMA_TIMEOUT_SECONDS=300
OLLAMA_TEMPERATURE=0.2
```